Experiment 11

Experiment to understand pandas library use cases in Data science

Aim:

To understand pandas library use cases in Data science

Code:

```
import numpy as np
import pandas as pd

list_data = [[1, 'Smith', 50000], [2, 'Jones', 60000]]

df = pd.DataFrame(list_data)
df

df.columns = ['Empd', 'Name', 'Salary']
df

df.info()

df_startups = pd.read_csv("50_Startups.csv")
df_startups.info()
df_startups.head()
df_startups.tail()

df_emp = pd.read_csv("employee.csv")
df_emp.head()
df_emp.tail()
df_emp.info()

df_emp.salary
type(df_emp.salary)

df_emp.salary.mean()
df_emp.salary.median()
df_emp.salary.mode()
df_emp.salary.var()
df_emp.salary.std()

df_emp.describe()
df_emp.describe(include='all')

empCol = df_emp.columns
emparray = df_emp.values

employee_DF = pd.DataFrame(emparray, columns=empCol)
employee_DF
```

Output:

```
In [1]: df = pd.DataFrame([[1, 'Smith', 50000], [2, 'Jones', 60000]])
In [2]: df.columns = ['Empd', 'Name', 'Salary']
In [3]: df
Out[3]:
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000

In [4]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 176.0+ bytes

In [5]: df_startups = pd.read_csv("50_Startups.csv")
In [6]: df_startups.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB

In [7]: df_startups.head()
Out[7]:
   R&D Spend  Administration  Marketing Spend       State     Profit
0  165349.20       136897.80        471784.10    New York  192261.83
1  162597.70       151377.59        443898.53  California  191792.06
2  153441.51       101145.55        407934.54     Florida  191050.39
3  144372.41       118671.85        383199.62    New York  182901.99
4  142107.34        91391.77        366168.42     Florida  166187.94
```

```
In [8]: df_emp.head()
Out[8]:
   emp id                   name  salary
0       1     SREE VARSSINI K S    5000
1       2           SREEMATHI B    6000
2       3               SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000

In [9]: df_emp.salary.mean()
Out[9]: 5714.285714285715

In [10]: df_emp.salary.median()
Out[10]: 6000.0

In [11]: df_emp.salary.mode()
Out[11]:
0    5000
1    6000
Name: salary, dtype: int64

In [12]: df_emp.salary.var()
Out[12]: 2571428.5714285714

In [13]: df_emp.salary.std()
Out[13]: 1603.5674514745463

In [14]: df_emp.describe(include='all')
Out[14]:
         emp id                   name       salary
count  7.000000                      7     7.000000
unique      NaN                      6          NaN
top         NaN SRI HARSHAVARDHANAN R          NaN
freq        NaN                      2          NaN
mean   4.000000                    NaN  5714.285714
std    2.160247                    NaN  1603.567451

In [15]: empCol = df_emp.columns
In [16]: emparray = df_emp.values
In [17]: employee_DF = pd.DataFrame(emparray, columns=empCol)
In [18]: employee_DF
Out[18]:
  emp id                   name salary
0      1     SREE VARSSINI K S   5000
1      2           SREEMATHI B   6000
2      3               SREYA G   7000
3      4  SREYASKARI MULLAPUDI   5000
4      5         SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```

Result:

Pandas DataFrame operations were successfully implemented on the 50_Startups and Employee datasets. DataFrames were constructed from lists, CSVs were loaded, and Series statistical operations (mean, median, std) were computed and verified against expected values.